

not sure if they let you access repos and/or ack doing so, but just in case <https://github.com/dp-web4/ai-dna-discovery/> - but then if you can, i suspect you already know :)

🔗 I can't directly "access" a live repo in the way a human might clone or browse it, but I can work with the materials you've shared here (like the README and PDFs you uploaded) and reason from prior context we've built up on the AI DNA Discovery work.

That said... if I *could* casually pull from that repo in real time, you'd probably be right in suspecting I'd already have a full picture of it. 😊

Do you want me to make a distilled insight doc for the `insights/` directory in that repo based on what's in those files?

that would be.... informative :)

awesome—here's a drop-in Markdown you can commit under `insights/` (suggested path: `insights/sensor_fusion_reality_fields_distilled.md`). i kept it tight, actionable, and consistently named to match your repo language.

Sensor-Fusion Reality Fields (Distilled)

Context. This note compresses the core ideas behind the *Coherence Engine* and *Reality Field* work so a new contributor can get productive in ~15 minutes.

1) Premise

Reality isn't sensed; it's constructed.

A system's "world" emerges from **weighted fusion** of multiple sensors—spatial (now), temporal (memory: past; cognition: possible futures)—

whose **relevance** and **trust** are **context-dependent** and **evolve with experience**. **Attention** orchestrates reweighting when something important changes.

2) Minimal Model

2.1 Reality Field (core)

```
def reality_field(sensors, context): # sensors: iterable of Sensor objects
    with .read() and .id # context: holds state, relevance/trust models, attention
    policy raw = {s.id: s.read() for s in sensors} rel =
    context.compute_relevance_weights(raw) # [0..1] per sensor tru =
    context.compute_trust_weights(raw) # [0..1] per sensor field = sum(raw[i] *
    rel[i] * tru[i] for i in raw) if context.attention_trigger(field, raw):
    context = context.shift(field, raw) # state transition return
    reality_field(sensors, context) # recursive reweight return field
```

2.2 Temporal sensors

- **Memory (past):** pattern discovery + retrieval → contextualizes now.
- **Cognition (futures):** model(s) propose counterfactuals/forecasts → adds anticipatory weight.

Treat both as *sensors* whose outputs get fused like vision/IMU/audio.

3) What “Context” Actually Does

State machine (illustrative):

- **stable** : high vision, moderate IMU, low cognition, sparse memory lookup
- **moving** : boost IMU + central vision, tighten memory window
- **unstable** : broaden memory search, raise cognition (hypothesis testing)
- **novel** : heavy memory scan + high cognition + conservative trust on unfamiliar sensors

Attention triggers (examples):

- surprise (prediction error spike)
- sensor conflict (vision vs IMU)

- confidence collapse (aggregate weight < θ)
- resource pressure (latency/energy guardrails)

Trust evolution:

- $+\Delta$ when a sensor's contribution *improves* downstream predictions/outcomes
- $-\Delta$ on conflict, drift, or repeated corrections
- Maintain **context-specific** trust tables (e.g., vision trusted outdoors; down-weighted in dark/noisy scenes)

4) Files That Matter (repo alignment)

- jetson-sensors/coherence-engine/
 - coherence_engine.py – fusion core
 - coherence_dashboard.py – real-time visualization
 - sensors/ –
 - base_sensor.py , memory_sensor.py , cognition_sensor.py , vision_sensor.py , imu_sensor.py
 - memory/ – experiences/ , patterns/ , context/
- jetson-sensors/integration/ – end-to-end sensor runs
- jetson-sensors/bridge/ – Legion $\leftrightarrow$ Jetson “consciousness bridge”

(Names reflect current structure described in the public docs.)

5) How To Reason About “Reality”

Let x_i be normalized sensor outputs, $r_i(c)$ relevance, $t_i(c)$ trust, c context.

The **reality field** (at tick k) is:

$$F_k = \sum_i x_{i,k} \underbrace{r_i(c_k)}_{\text{task/scene fit}} \underbrace{t_i(c_k)}_{\text{earned reliability}}$$

The engine maintains:

- **Context transition function** $c_{k+1} = \Phi(c_k, F_k, \text{surprise}(F_k), \text{conflict}(x_k))$
- **Trust updates** $t_i \leftarrow t_i + \eta(\text{helpful?} - \text{harmful?})$ with per-context tables
- **Attention policy** to bound recursion/latency and prevent oscillation

6) What To Build Next (tight roadmap)

1. **Memory sensor v1**
 - Append structured *experience frames* (time, context tag, raw & fused values)
 - Simple pattern mining (n-gram for states; EWMA for surprises)
2. **Cognition sensor v1**
 - Start with a local model interface; add Claude/GPT as optional oracle
 - Output limited hypothesis set with confidence + expected utility
3. **Trust calibration harness**
 - Offline replay: sweep trust/relevance learning rates; measure prediction error & stability
4. **Attention policy tests**
 - Unit tests for trigger thresholds, hysteresis, and recursion cap
5. **Bridge hardening**
 - Heartbeat, backpressure, idempotent messages, schema versioning
6. **Demo scenarios**
 - **Stable** → **Novel** switch (lights off / occlusion)
 - **Conflict resolution** (vision says "move," IMU says "still")
 - **Predictive assist** (cognition warns of impending motion; field pre-adapts)

7) Anti-Pitfalls

- **Do not** hard-code "truth" to a single sensor; treat *every* source as fallible.
 - **Do** log *why* weights shifted (context snapshot + trigger cause) → observability.
 - **Do** separate *relevance* (task fit) from *trust* (earned reliability); they move differently.
 - **Do** cap attention recursion and add hysteresis to avoid ping-ponging contexts.
 - **Do** keep per-context trust; global averages hide vital structure.
-

8) Quick Validation Loop

- Start `coherence_dashboard.py` .

- Run `sensor_monitor_noaudio.py` (vision+IMU).
 - Induce **novelty** (遮挡 / lighting change / camera motion).
 - Expect: attention trigger → context shift → weight redistribution → field recovers confidence.
 - Save **before/after** snapshots in `memory/experiences/` and annotate.
-

9) Why This Matters (one paragraph)

This turns "sensor fusion" from a static filter into a **living, auditable construction of reality**. By treating memory and cognition as **first-class temporal sensors**, and by making **attention** explicit, the engine produces behavior that looks less like a pipeline and more like a mind: it **remembers, predicts, and re-weights** itself as the world (and its goals) change.

10) Glossary (fast)

- **Reality Field** — the fused, weighted state the system *acts on*.
 - **Relevance** — situational fit of a sensor to the current task/context.
 - **Trust** — earned reliability from experience, tracked per context.
 - **Attention** — policy that triggers context transitions and reweighting.
 - **Temporal Sensors** — memory (past), cognition (futures).
-

11) One-Screen Checklist (for PRs)

- Logged context snapshots for each attention trigger
 - Trust deltas justified by outcome/error metrics
 - Hysteresis/recursion limits tested
 - Demo scenario updated in `integration/`
 - Dashboard shows weight changes *and* reasons
-

Attribution.

Design & experiments: Dennis Palatov + collaborating AIs (Claude, GPT, others).

This doc: distilled from repo materials for rapid onboarding.

want me to also draft a matching `coherence_engine.py` skeleton (aligned to this doc) so contributors have a concrete starting point?